LineTR End-to-End Implementation Plan Report

System Architecture & Engineering Plan

Abstract

This report outlines the end-to-end implementation strategy for deploying the LineTR text line segmentation model tailored for our specific use case (e.g., historical Modi script manuscripts). To maintain a scalable, modular, and maintainable codebase, the architecture is strictly separated into interacting Python modules rather than monolithic scripts or notebooks. The implementation is broken down into six sequential phases. **Note: This document provides conceptual and architectural blueprints with strictly no code.**

Contents

1 Introduction and Architecture Principles

The objective of this project is to implement LineTR from scratch. The system is designed to handle high-resolution, complex handwritten manuscripts without downsampling. To achieve this, the system uses a two-stage approach: predicting line structures on dynamically sized patches (Stage 1) and generating polygonal bounds via seam generation (Stage 2).

Modularity Principle: The project will be divided into specific Python modules (e.g., configurations, data loaders, neural network blocks, loss calculations, and orchestration).

—

2 Phase 1: Data Preparation & Context-Adaptive Patching

2.1 Concept

Instead of resizing an entire document and losing dense text features, LineTR relies on extracting patches. Because text scale varies, the patch size must be adaptive. The system calculates an expected inter-line gap from a sample of patches, computes a global average, and then defines a fixed "context-adapted" patch size for the entire document.

2.2 Implementation Plan

- **Proposed Module:** `data_processing.py`
- **Data Ingestion:** Implement functions to load high-resolution manuscript images and normalize their color channels.
- **Dynamic Sampler:** Implement an algorithm that samples candidate patches at various predefined scales, interfaces with the initial model forward-pass to measure line gaps, and computes the optimal patch size dimension t.
- **Patch Extraction:** Create a sliding-window grid generator that extracts overlapping $t \times t$ patches and resizes them uniformly to 256×256 for neural network consumption.
- **Data Loaders:** Build a standard dataset and dataloader interface that can yield these patches in batches during both training and inference.

—

3 Phase 2: Core Network Architecture (Stage-1 Models)

3.1 Concept

Stage 1 represents the deep learning core, consisting of three primary components: a shared Vision Transformer (ViT) backbone, a DETR-style Line-Parameter Generator,

and a hybrid CNN-Transformer Text-Energy Map Generator.

3.2 Implementation Plan

- **Proposed Modules:** `models.py`, `vit_backbone.py`, `line_generator.py`, `energy_generator.p`
- **Shared Backbone (`vit_backbone.py`):** Construct a standard ViT architecture configured for 16×16 patch sizes to output a specific embedding dimension.
- **Line-Parameter Generator (`line_generator.py`):**
  - Define the learnable line queries and positional priors.
  - Construct the transformer decoder layers, specifically separating self-attention and decoupled row-column cross-attention to process the ViT features.
  - Attach a lightweight feed-forward head that outputs the normalized coordinates (μ_x, μ_y), slope (m), and confidence probability.
- **Text-Energy Map Generator (`energy_generator.py`):**
  - Construct a CNN encoder that iteratively downsamples the input.
  - Build a transformer decoder module that cross-attends the ViT backbone features with the CNN features.
  - Construct a CNN decoder with skip connections linking back to the CNN encoder to reconstruct spatial resolution.
  - Apply a Sigmoid activation layer at the end to output a continuous $[0, 1]$ map.

—

4 Phase 3: Loss Formulations & Training Pipeline

4.1 Concept

Because LineTR predicts a set of un-ordered geometric lines rather than pixel classifications, it requires bipartite matching (Hungarian Algorithm) to pair ground-truth lines with model predictions before computing losses. The training is conducted in two distinct phases to ensure stability.

4.2 Implementation Plan

- **Proposed Modules:** `losses.py`, `trainer.py`
- **Loss Formulations (`losses.py`):**
  - Implement the Bipartite Matching cost function utilizing distance geometry.
  - Implement the Geometric Loss function, which computes the squared vertical distances at the left and right boundaries of the patch.
  - Implement the Focal Loss function to penalize false positives and missed line classifications.

- **Training Orchestration (`trainer.py`):**
  - *Training Phase 1:* Define a training loop that exclusively updates the weights of the ViT Backbone and Line-Parameter Generator over a set number of epochs.
  - *Training Phase 2:* Define a subsequent loop that freezes the aforementioned models and exclusively trains the Text-Energy Map Generator.
  - Integrate learning rate schedulers and optimizer configurations (e.g., AdamW).

—

5 Phase 4: Post-Processing & Global Stitching

5.1 Concept

During inference, the model generates outputs for hundreds of isolated, overlapping patches. These localized predictions must be seamlessly mapped back into the coordinate space of the original high-resolution document.

5.2 Implementation Plan

- **Proposed Module:** `post_processing.py`
- **Scribble Un-normalization:** Implement a function to reverse the coordinate normalization, mapping patch-level midpoints and slopes to document-level pixel coordinates.
- **Iterative Projection-Merging:** Implement a clustering algorithm that groups nearby, overlapping line segments (scribbles) based on spatial proximity and slope similarity, merging them into unified, global text-line scribbles.
- **Energy Map Stitching:** Implement a blending mechanism to stitch the predicted 256×256 continuous energy patches together, handling the overlapping boundaries to output a single, smooth, high-resolution global energy map.

—

6 Phase 5: Stage-2 Seam Generation

6.1 Concept

The global scribbles indicate *where* the text is, and the continuous energy map indicates the *density/presence* of the text. Seam Carving algorithms are used to find the optimal pathways (valleys of low energy) between the scribbles to draw precise polygonal boundaries around the text lines.

6.2 Implementation Plan

- **Proposed Module:** `seam_carving.py`
- **Cost Matrix Construction:** Use the continuous energy map directly (without thresholding) to build a cumulative energy traversal matrix.
- **Dynamic Programming Seam Search:** Implement the core seam-finding logic. Condition the search boundaries using the global scribbles so that the algorithm reliably finds the medial, upper, and lower seams.
- **Polygon Extraction:** Convert the generated seams into closed, distinct polygon coordinates that can be exported or plotted over the original image.

—

7 Phase 6: Orchestration & Inference Pipeline

7.1 Concept

The final phase ties all modular components together into an automated, seamless pipeline that can accept a raw image and output final polygonal segmentation coordinates.

7.2 Implementation Plan

- **Proposed Modules:** `main.py`, `config.yaml`
- **Configuration Management (`config.yaml`):** Store all hyperparameters (patch sizes, neural network dimensions, learning rates, loss weights).
- **End-to-End Orchestrator (`main.py`):**
  - Load configurations and instantiate the models.
  - Load a document via `data_processing.py`.
  - Execute the dynamic patch generation and feed patches through `models.py`.
  - Route the neural outputs through `post_processing.py`.
  - Pass the global maps into `seam_carving.py` to finalize the polygons.
  - Save or visualize the final segmentation results.